

Fase 4 — Fluxo de Compra Transacional (Carrinho, Pedido, Saga e Mensageria)

Projeto: **ecommerce-platform** | Repositorio: github.com/DaviMoraismdev/ecommerce-platform |
Status: **Fase 4 concluida** (Bloco 10 — fechamento)

1. Visao geral da Fase 4

A Fase 4 fechou o **nucleo comercial** da plataforma: o caminho completo do cliente colocando itens no carrinho ate o pedido ser confirmado, ter estoque reservado de forma atomica e disparar eventos consumidos por outro servico — tudo com garantias de consistencia sob concorrencia e falha.

O foco nao foi adicionar mais servicos, e sim provar que os servicos conversam de forma **correta**. Tres propriedades guiaram todas as decisoes:

- **Nao vender o que nao existe** — reserva de estoque atomica, sem oversell mesmo com requisicoes concorrentes.
- **Nao cobrar/processar duas vezes** — idempotencia em toda operacao que causa efeito (checkout e consumo de eventos).
- **Nao perder eventos** — publicacao confiavel via outbox transacional, com entrega *at-least-once* e tratamento de mensagens venenosas por DLQ.

O resultado e validado **ponta a ponta** por uma suite e2e que sobe a stack real e exercita os fluxos felizes e os de falha (oversell, replay, JWT expirado, DLQ).

Item	Resultado
Servicos tocados	cart, order, inventory, notification (+ product/auth como dependencias)
Blocos entregues	1 a 10 (implementacao, testes e documentacao)
Padrao de entrega	branch por bloco → Pull Request → revisao externa → squash merge
Cobertura de testes	unitarios + integracao por servico + suite e2e da stack completa
Garantias provadas	sem oversell, idempotencia de checkout, entrega at-least-once, DLQ

2. O que foi construido (blocos 1 a 10)

Cada bloco e uma unidade fechada: implementacao, teste com output real e Pull Request revisado antes do merge. Abaixo, o mapa da fase.

2.1 Carrinho (cart-service)

Servico de carrinho sobre **Redis** (ioredis), com operacoes de adicionar, remover e atualizar itens. O carrinho guarda o essencial (produto + quantidade) e **enriquece o preco em tempo de leitura** consultando o product-service, em vez de congelar o preco no Redis — isso evita servir preco desatualizado. Inclui validacao de estoque, chave por usuario e TTL para expiracao de carrinhos abandonados. Quando um item nao pode ser resolvido, o carrinho retorna estado *parcial* em vez de falhar por inteiro.

2.2 Pedido (order-service) — o nucleo transacional da fase

E aqui que mora a maior parte da complexidade. O checkout e uma **saga**: uma sequencia de passos entre servicos em que cada passo tem uma **compensacao** caso um passo seguinte falhe. O pedido nasce, reserva estoque no inventory, e — se algo falhar depois — a reserva e liberada (compensada).

- **Idempotencia por Idempotency-Key**: o mesmo checkout reenviado nao cria dois pedidos. A chave e reivindicada *antes* dos efeitos (claim-before-effects).
- **Total recalculado no servidor**: `total = Σ subtotais`, nunca confiando no valor que veio do cliente.
- **Maquina de estados**: PENDENTE → PAGO → ENVIADO → ENTREGUE (+ cancelamento), com transicoes protegidas por *compare-and-swap* para evitar corrida de status.
- **Outbox transacional + relay**: o evento de dominio e gravado na *mesma transacao* do pedido; um relay faz o polling e publica no broker — entrega *at-least-once*.

Cobertura: **54 testes unitarios + 31 de integracao + e2e**.

2.3 Estoque (inventory-service) — ownership da reserva

A reserva de estoque passou a ter **dono**: cada reserva e amarrada a um `orderId`, e o `release(orderId)` e **idempotente** — liberar duas vezes nao devolve estoque a mais. Isso e o que permite a saga do order-service compensar com seguranca. A deducacao de estoque usa operacao atomica (*compare-and-swap* no banco) para nunca ficar negativa sob concorrencia.

2.4 Eventos e mensageria (order → notification)

Com o pedido publicando eventos de forma confiavel, o **notification-service** passou a consumi-los de forma resiliente:

- Consome `order.created` e `order.status_changed`.
- **Idempotencia por eventId** com Redis (SET NX PX): reivindica o evento com um *token* de posse e libera com um *compare-and-delete* em Lua — garante que so quem reivindicou pode liberar.
- **Dead-letter queue (DLQ)**: mensagem que falha o processamento apos as tentativas vai para uma fila morta, em vez de travar o consumo (poison message).
- **Shutdown gracioso** e requeue com atraso. Cobertura: **39 testes unitarios**.

Importante: esta fase entregou a **infraestrutura** de notificacao (consumo confiavel de eventos). O **efeito real** — enviar e-mail/push ao cliente — permanece como escopo da Fase 6; hoje a acao do consumer e um ponto de extensao, nao uma entrega.

2.5 Testes ponta a ponta (e2e) — Bloco 9

Uma suite e2e separada (Jest + fetch global + timeout por AbortController) sobe a stack real e valida o fluxo do usuario de verdade: registro/login, catalogo, carrinho, checkout, reserva de estoque e notificacao. Cobre tambem os **caminhos de falha**: oversell sob concorrencia, replay de checkout apos 409, JWT expirado e assinatura invalida.

Guarda de seguranca: a suite fabrica JWTs de ADMIN e cria dados reais, entao por padrao so roda contra `localhost`. Rodar contra alvo `nao-local` exige `E2E_ALLOW_DESTRUCTIVE=true` explicitamente.

2.6 Fechamento (Bloco 10) — documentacao

Auditoria do `TECH_DEBT.md` (mantendo so o que falta, cada pendencia com destino explicito — Fase 7, Fase 10 ou gatilho), atualizacao do `README.md` para a estrutura atual, confirmacao dos servicos

fechados e esta revisao em PDF.

3. Conceitos consolidados

Os conceitos abaixo deixaram de ser teoria e viraram código testado nesta fase.

3.1 Saga com compensacao

Transacao distribuida nao tem ROLLBACK global. A saga resolve isso quebrando a operacao em passos locais, cada um com uma acao de compensacao. Se o passo N falha, voce executa as compensacoes dos passos 1..N-1 na ordem inversa. No checkout: se a reserva de estoque sucede mas um passo posterior falha, a compensacao e `release(orderId)`.

3.2 Idempotencia (claim-first)

Uma operacao idempotente pode ser repetida sem mudar o resultado alem da primeira vez. O padrao usado e **reivindicar antes de causar efeito**: grava-se a chave (Idempotency-Key no checkout; eventId no consumer) *antes* de executar. Se a chave ja existe, retorna-se o resultado anterior em vez de reprocessar.

```
# Consumer: reivindica o evento antes de agir
SET notif:proc: NX PX 60000 # so cria se nao existe
# -> se 'OK' : eu ganhei a posse, posso processar
# -> se null : alguem ja processou -> descarta (idempotente)

# Ao terminar, libera SO se o token ainda for meu (Lua compare-and-delete):
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('DEL', KEYS[1])
end
return 0
```

3.3 At-most-once vs at-least-once

Sao as duas garantias de entrega possiveis na pratica. **At-most-once**: a mensagem chega no maximo uma vez (pode se perder). **At-least-once**: a mensagem chega pelo menos uma vez (pode duplicar). Sistemas confiaveis escolhem **at-least-once** e resolvem a duplicacao com idempotencia no consumidor — foi exatamente a combinacao adotada aqui: outbox garante que o evento sai; idempotencia por eventId garante que processa-lo duas vezes nao faz mal.

3.4 Transactional outbox + relay

O problema classico: como gravar no banco e publicar no broker sem que um suceda e o outro falhe? A solucao e nao publicar direto. Grava-se o evento numa tabela outbox **na mesma transacao** do dado de negocio. Um processo a parte (o *relay*) faz polling da outbox e publica no broker, marcando como enviado. Se o publish falha, o relay tenta de novo — dai o at-least-once.

```
// Grava pedido + evento ATOMICAMENTE (Prisma $transaction)
await prisma.$transaction([
  prisma.order.create({ data: order }),
  prisma.outbox.create({ data: { type: 'order.created', payload, sent: false } }),
]);
// Relay (processo separado, em loop):
// 1. le outbox WHERE sent = false
// 2. publica no broker
// 3. marca sent = true (se falhar aqui, reenvia -> duplica -> consumer deduplica)
```

3.5 Dead-letter queue (DLQ)

Uma mensagem que sempre falha (payload invalido, bug) nao pode bloquear a fila para sempre nem ser reprocessada infinitamente — e uma *poison message*. A DLQ e o destino dessas mensagens: depois de esgotar as tentativas, a mensagem e desviada para uma fila morta (via dead-letter exchange), onde pode ser inspecionada sem travar o consumo normal.

3.6 Compare-and-swap e reserva atomica

Para nunca vender estoque a mais sob concorrência, a dedução usa **compare-and-swap**: a atualização só acontece se a condição ainda for verdadeira (*há estoque suficiente*) no instante da escrita, tudo numa operação atômica. Duas requisições simultâneas para o último item: uma vence, a outra recebe conflito — nunca as duas.

```
-- So reserva se ainda houver saldo; a condicao e a escrita sao atomicas
UPDATE inventory
  SET available = available - :qty
  WHERE product_id = :id
  AND available >= :qty;      -- 0 linhas afetadas = sem estoque = 409
```

4. Comandos aprendidos

Comandos e padroes de terminal que passaram a fazer parte do fluxo nesta fase.

Prisma (inventory / order)

```
npx prisma migrate dev --name      # cria e aplica migracao em DEV
npx prisma migrate deploy          # aplica migracoes pendentes (CI/prod)
npx prisma generate                # regenera o client tipado
npx prisma studio                  # inspeciona dados no navegador
```

Jest / testes

```
npm test                          # unit + integracao do servico
npx jest --watch                  # re-roda ao salvar
jest.useFakeTimers()              # controla setTimeout/relay nos testes
cd e2e && npm ci && npm test      # suite ponta a ponta (stack de pe)
```

RabbitMQ / mensageria

```
# DLX: fila principal com dead-letter exchange configurado
assertExchange('notifications.orders.dlx', 'fanout', { durable: true })
assertQueue('notifications.orders.dlq', { durable: true })
# fila de trabalho aponta o x-dead-letter-exchange para a DLX acima
```

Redis (idempotencia)

```
SET chave valor NX PX 60000      # cria so se nao existe, expira em 60s
EVAL 1 chave token                # compare-and-delete atômico (libera posse)
redis-cli MONITOR                  # observa comandos em tempo real (debug)
```

Docker Compose (stack local)

```
docker compose up -d             # sobe postgres, mongo, redis, rabbitmq
docker compose ps                 # confere servicos de pe
docker compose logs -f            # acompanha logs
docker compose down -v            # derruba e APAGA volumes (reset limpo)
```

5. Historico de problemas e solucoes

Erros reais enfrentados na fase, com sintoma, causa, correção e o porque da correção funcionar. O objetivo é reconhecer o padrão quando ele reaparecer — não apenas ter aplicado o comando que

resolveu.

5.1. Heredoc gravava barra invertida literal no código

Sintoma: Arquivos criados por heredoc apareciam com `\'` e `\n` literais no lugar de aspas e quebras de linha, quebrando o TypeScript.

Causa: Escapes escritos dentro de uma string Python de aspas simples (`' ' ' . . . ' ' '`) são interpretados como barra literal, e o texto ia cru para o arquivo.

Correção: Passar a usar heredoc Python (`python3 - << 'PYEOF'`) com aspas simples no delimitador e quebras de linha reais, sem escapar nada.

Por que funciona: Com o delimitador entre aspas simples o shell não interpola, e sem os escapes manuais o conteúdo chega ao arquivo exatamente como escrito.

5.2. Token JWT contaminado por mensagem do dotenv

Sintoma: Um token gerado por `node -e` vinha com texto extra e falhava a verificação de assinatura.

Causa: O `dotenv` v17 imprime uma dica (*injected env*) no **stdout**, e o `$(node -e ...)` capturava essa linha junto com o token.

Correção: Configurar `dotenv` com `{ quiet: true }` e ainda filtrar a saída com `| tail -n1` para pegar só a última linha.

Por que funciona: Silenciar a dica remove a fonte do ruído; o `tail -n1` é uma garantia extra de que só o token (última linha) é capturado.

5.3. 406 PRECONDITION_FAILED ao declarar a fila

Sintoma: O broker recusava a inicialização do consumer com 406 `PRECONDITION_FAILED`.

Causa: A fila durável já existia com um conjunto de argumentos; tentar redeclará-la com argumentos diferentes é proibido pelo RabbitMQ.

Correção: Apagar a fila uma única vez manualmente e simplificar o `assertQueue` para os argumentos definitivos.

Por que funciona: A declaração de fila é idempotente *somente* se os argumentos batem; alinhando o `assert` ao estado real, futuras subidas passam a ser inofensivas.

5.4. TS5011 e client Prisma desatualizado após generate

Sintoma: Erros de compilação (TS5011) e tipos do Prisma não reconhecidos mesmo depois de rodar `prisma generate`.

Causa: Configuração de `rootDir` no `tsconfig` de dev e cache do TS Server apontando para o client antigo.

Correção: Ajustar o `rootDir` e reiniciar o TS Server no editor.

Por que funciona: O client tipado é regenerado em disco, mas o servidor de linguagem só o recarrega ao reiniciar — o erro era de cache, não de código.

5.5. Marker de notificacao desaparecia (consumer)

Sintoma: A chave de idempotencia (`notif:proc:...`) as vezes sumia e o evento parecia nao processado.

Causa: A gravacao do marker podia estourar timeout e a falha de *release* nao era tratada, deixando o estado inconsistente.

Correcao: Envolver `recordProcessed` em `withTimeout` e rotear falha de *release* para a **DLQ**, com log sanitizado.

Por que funciona: Toda operacao de rede passa a ter tempo limite e caminho de erro definido; mensagem problematica vai para a DLQ em vez de corromper o fluxo normal.

5.6. Qualquer processo podia liberar a posse do evento

Sintoma: Apontado em revisao: um `DEL` simples no *release* permitia que um processo apagasse a posse de outro.

Causa: Sem verificar dono, o *release* nao distinguia quem realmente reivindicou o evento — risco de reprocessamento concorrente.

Correcao: Guardar um **token** de posse no `SET NX` e liberar com um *compare-and-delete* em Lua (so apaga se o token bater).

Por que funciona: A checagem e a exclusao acontecem atomicamente no Redis, entao so o dono legitimo remove a chave — elimina a corrida entre consumidores.

6. Definition of Done e proximos passos

6.1 Checklist de conclusao da Fase 4

- ✓ Funciona localmente sem erros nao tratados (stack completa de pe).
- ✓ Verificado com output real — unit, integracao e e2e passando.
- ✓ Sem segredo exposto — `.env` sempre no `.gitignore`; so `.env.example` versionado.
- ✓ Documentado — README, TECH_DEBT e esta revisao em PDF atualizados.
- ✓ Commits no padrao Conventional Commits; merge via Pull Request revisado.
- ✓ Davi consegue explicar o que foi feito e por que (saga, idempotencia, outbox, DLQ).

6.2 Dividas em aberto (com destino explicito)

Nenhuma divida foi deixada sem endereco. Resumo do que ficou registrado no `TECH_DEBT.md`:

Divida / evolucao	Destino
mTLS / identidade de servico (hoje order assina JWT ADMIN com segredo compartilhado)	Fase 7 (Seguranca)
DLQ com parking queue e classificacao de falhas	Fase 7 (Infra)
Backoff / quarentena / redrive automatizado no relay	Fase 10 (Jobs)
Reconciliacao e arquivamento/cleanup da outbox	Fase 10 (Jobs)
Efeito real do consumer (e-mail/push) e crash pos-claim	Gatilho: evolucao do notification (Fase 6+)
CI/CD (pipeline de build e testes)	Fase 7 (Deploy)

6.3 Proximos passos

Com o núcleo comercial fechado e testado, a sequência natural é:

- **Fase 5** — pagamento (payment-service) e/ou usuário (user-service), completando o domínio de negócio.
- **Fase 6** — evolução do notification para efeito real (e-mail/push).
- **Fase 7** — segurança e infraestrutura: gateway (Nginx), mTLS, CI/CD, DLQ operacional.
- **Fase 10** — jobs de reconciliação, cleanup e hardening de resiliência.

7. Visão geral do projeto hoje

Panorama do estado da plataforma ao fim da Fase 4.

7.1 Serviços

Serviço	Papel	Stack	Status
auth-service	Autenticação, JWT, RBAC	PostgreSQL	Concluído
product-service	Catálogo de produtos	MongoDB + Redis (cache)	Concluído
inventory-service	Estoque, reserva atômica com ownership	PostgreSQL / Prisma	Concluído (Fase 4)
cart-service	Carrinho, enriquecimento de preço	Redis (ioredis)	Concluído (Fase 4)
order-service	Checkout, saga, outbox, máquina de estados	PostgreSQL / Prisma	Concluído (Fase 4)
notification-service	Consumo de eventos, idempotência, DLQ	RabbitMQ + Redis	Parcial
user-service	Perfil / dados do usuário	a definir	Planejado
payment-service	Pagamento	a definir	Planejado
gateway (Nginx)	Ponto único de entrada	Nginx	Planejado
frontend	Interface do cliente	a definir	Planejado

Nota: o notification-service está **Parcial** — a infra de consumo (idempotência, DLQ, shutdown gracioso) foi entregue na Fase 4; o efeito real de notificação (e-mail/push) e escopo da Fase 6.

7.2 Fluxo comercial (o que já funciona ponta a ponta)

```
Cliente
| registro/login (auth)          -> JWT
| navega catálogo (product)     -> preço atual
| adiciona ao carrinho (cart)    -> Redis, valida estoque
v checkout (order)
  |-- reserva estoque (inventory) [atômico, sem oversell]
  |-- grava pedido + evento outbox [mesma transação]
  |-- relay publica no RabbitMQ   [at-least-once]
  v
    notification consome o evento [idempotente + DLQ]
```

7.3 Onde está a maturidade

O que está **solido**: consistência sob concorrência, idempotência ponta a ponta, publicação confiável de eventos e cobertura de testes em três níveis (unit, integração, e2e). O que é **consciente e adiado** (não esquecido): identidade de serviço via mTLS, gateway, CI/CD e jobs de reconciliação — todos com fase de destino registrada. Nenhuma decisão de segurança foi tomada por acidente; a única exceção aceita

(order assinando JWT ADMIN com o segredo compartilhado) esta documentada com justificativa e correcao prevista.

Revisao da Fase 4 — projeto ecommerce-platform. Documento gerado no fechamento do Bloco 10. Proxima revisao: fim da Fase 5.